

Demo script: Fraud Detection AI Agent

Anand Kumar Singh · IIT Bhilai · 2024–2026

A spoken script for a 10-minute demo, with what to click in **bold**. Timings are a guide. Everything in it is measured from this project, not estimated.

Before you start

- Run `deploy/open_dashboard.sh` (or `uv run fraud-api`) and open the console.
- Have one alert with a finished case ready to show: **CLM000078** (critical, fake documents) or **CLM000029** (exaggerated amount).
- Pick one open alert to investigate live. It takes about three minutes, so start it early and talk over it, or say you will return to it.
- Optional: have the BigQuery console open on the `fraud_cases` dataset.

1. What the project does (1 min)

Insurance fraud does not happen in one place. It happens when a policy is sold, when the premium is paid, and when a claim is made — and it is committed by customers, by agents, and by people pretending to be agents.

This project is a system that finds suspected fraud automatically, investigates it, and hands a human investigator a written case with the evidence already checked. The person still decides. The system does the searching, the cross-checking and the writing.

I built it on four labelled datasets of 1,000 records each, on Google Cloud, with a team of eight AI agents doing the investigation.

2. The data: four CSV files (1.5 min)

Show: slide 3 of the deck, or the `fraud_clean` dataset in BigQuery.

The data is synthetic but realistic, modelled on the Indian insurance market: amounts in rupees, Indian states, UPI and NEFT payments. Four files, 1,000 records each, each row labelled fraud or not, and if it is fraud, which of 17 types.

File	What it is	Fraud
<code>policies_free_insurance.csv</code>	One row per policy. Fraud here is getting cover without really paying: fake policies with no premium collected, backdated start dates, fronting (the named holder is a front for someone else), and free-look abuse (cancelling inside the 15-day refund window).	15.5%
<code>claims.csv</code>	One row per claim. Duplicate claims, staged accidents, forged documents, inflated amounts, and claims made suspiciously soon after the policy starts.	18.4%
<code>payments_bad_payments.csv</code>	One row per premium payment. Money that never really reached the insurer: unauthorised refunds, fake receipts, bounced cheques used to get cover, and agents keeping the premium.	14.4%

File	What it is	Fraud
ghost_broking.csv	One row per sale by an agent. "Ghost broking" is selling insurance without the right to: fake or cloned agent identities, expired or suspended licences, no licence at all, and agents pocketing premiums.	28.5%

The files are linked by three keys — policy, customer and agent — which is what makes cross-file investigation possible. 222 agents appear in all four files.

One important detail: the labels are used only for training and for scoring the system. The agents and the console never see them, so when I quote accuracy, the system earned it without looking at the answers.

3. The approach and architecture (2 min)

Show: slide 6 of the deck (four layers), or draw it.

The design principle is that each technique does what it is best at.

Rules catch the certain cases: a receipt number used twice, a payment that bounced, a premium never remitted. They are exact, instant and free, and every hit explains itself. There are 19 of them, and their severity is not guessed: it comes from measured precision on the labelled data.

A machine learning model scores claims, because claims are where the rules are weakest. Gradient-boosted trees, 29 features drawn from all four files.

AI agents investigate what the first two layers flag. This is where they earn their place: gathering context across four files, weighing it, and writing it up.

A human decides. Every decision is stored and becomes a new label for retraining.

Underneath, one data store: BigQuery, in four datasets that mirror those layers — raw, cleaned, features, and cases. Every layer reads and writes there, so any stage can be re-run on its own.

4. Why agents, and where they are used (1.5 min)

A fair question is: why not just ask a language model to read every row?

Because it would be slow, expensive and unreliable at that job. There are 4,000 records; rules and a model screen all of them in seconds. Only 778 became alerts, and only those go to the agents. The expensive tool is spent where its strength matters.

There are eight agents, built with Google's Agent Development Kit:

- an **orchestrator** that takes an alert and routes it;
- four **specialists**, one per file: claims, policy, payment and ghost broking;
- a **network agent** that looks for fraud rings around an agent or customer;
- a **case writer** that turns the findings into a structured case file;
- a **reviewer** that re-checks the case.

The reviewer is the part I would point to. It never sees the investigator's notes. It takes the finished case, re-queries every cited record in BigQuery, and marks each fact verified, contradicted or unverifiable. It also corrects over-called risk. Across the live runs it checked 72 facts: 71 verified, one unverifiable because of a gap in a tool, none contradicted. It has made real corrections — for example removing fraud types that described the agent rather than the record being investigated.

Three guards keep the agents honest: their tools are read-only, every query is parameterised, and the tools never return the label columns. Each specialist can only name fraud types that belong to its own file, and that is enforced by the output format, not by a polite instruction.

5. Why BigQuery (1 min)

I started with DuckDB, which is a local file-based database, and moved to BigQuery deliberately.

The deciding reason is that this is a deployed, multi-user system. The agents, the API and the console all need the same data at the same time, and the console runs on Cloud Run, where a local file would have to be copied into every container. Several containers cannot safely write to one file; BigQuery handles concurrent access.

Three more reasons. Access control: the service account can read the cleaned data but only write cases and decisions, and it cannot see the raw dataset at all. Audit logs, which matter for fraud work. And data residency: everything sits in the Mumbai region.

At 4,000 rows DuckDB would be faster. At production volumes BigQuery does not change shape, and the rules already run as SQL inside it.

6. Tech stack (1 min)

Layer	Choice
Language	Python 3.12, managed with uv
Data	Google BigQuery, Mumbai region
Rules	Conditions in a YAML file, compiled to SQL and run inside BigQuery
ML	scikit-learn gradient boosting, tested on agents it never saw
Agents	Google ADK 2.9 on Vertex AI, Gemini 3.7 Flash
API	FastAPI, with long agent runs as background jobs
Console	React 18 with TypeScript, built by Vite
Deployment	Cloud Run, private, scales to zero
Quality	41 automated tests

Two notes a reviewer may appreciate. The rules live in YAML, so a fraud analyst can change a threshold without touching code. And the tests exist mainly to prevent one specific mistake: using a label or an investigation outcome as a feature. A test fails if anyone reintroduces one.

7. Live demo (3 min)

Open the console.

This is the investigator's console. 778 alerts, highest priority first. Each row shows the record, its severity, how it was flagged, and its status: open, case ready, or decided.

Filter to "case ready" and open CLM000078.

Here is a finished case. At the top: the risk after review — critical — the recommended action, and the fraud type the agents named. Below that the reviewer's verdict and note.

Then the part I care about most: the evidence table. Every line is a fact the agents asserted, with the record it came from, and a mark showing the reviewer checked it against the data. Six facts, all verified. This claim really was labelled "fake supporting documents", and the agents found it without ever seeing that label.

Below: next steps for the investigator, open questions the data cannot answer, and the people and policies involved. If the agent behind the record is a network hub, the network analysis appears here too.

Scroll to the decision form. Record a decision, or explain it.

The investigator confirms fraud, dismisses it, or asks for more information. That decision is written to BigQuery and becomes a new label for retraining. That is the loop closing.

Open an alert with no case and press "Investigate now".

For an alert with no case yet, the agents can investigate on demand. It shows what they are doing — the specialist, then the case writer, then the reviewer — and takes about three minutes.

8. Results, honestly (1.5 min)

Rules alone catch 100% of the bad payments and 95% of the ghost-broking fraud, with no false alarms on the certain rules. They are weakest on claims: 29%.

The claims model raises that to 35% at over 80% precision. That is a modest gain, and I want to be straight about why: three claim fraud types — staged accidents, duplicate claims and fronting — leave no trace in this data. 51% of honest claims also have a related claim within 180 days. No model can find what is not there.

The network analysis produced the most interesting result. 51 agents are tied to 5 or more alerts across at least 2 files. Records of those agents that nothing flagged are fraud 12.3% of the time, against 2.8% for everyone else — 91 fraud records the screening missed, and a place for an investigator to look.

The agents are good at explaining and weak at deciding. On a held-back set they did not beat the rules at classification, so I stopped tuning them for that and gave them the job they are good at: gathering evidence and writing the case. Rules and the model decide what gets flagged; the agent's risk rating is advice.

And the caveat that applies to all of it: the data is synthetic. The pipeline transfers to real data; these accuracy numbers do not.

9. Engineering: making it fast (1 min, optional)

An investigation originally took over 13 minutes. It now takes three.

I measured instead of guessing. A one-word prompt to the model took 81 seconds, which pointed at throttling rather than the code. Current Gemini models on Vertex AI run on dynamic shared quota, so there is no per-project limit to raise.

Three changes: I merged each tool's several BigQuery queries into one and ran the rest concurrently; I gave the agents a single prefetched bundle instead of four separate lookups, removing three model round trips per stage; and I gave the reviewer a bulk lookup, which took that stage from 13 minutes to under a minute. Then I benchmarked the models and found the one I was using was seven times slower than its neighbours on shared capacity, so I switched.

I also tested a lighter model. It stalled on two of four alerts and over-called fraud types, with no real speed gain, so I kept the current one. The reviewer caught the over-calling, which was a good sign for the design.

10. Closing (30 s)

To summarise: cheap exact methods screen everything, AI agents investigate what is flagged and must cite their evidence, a second agent verifies that evidence against the database, and a person decides. The measurements are honest because the labels were hidden from the system throughout.

Next steps: real data with richer signals such as repair-shop and hospital identifiers, retraining from investigator decisions, and a scheduled daily run.

Likely questions

Why not train one big model on everything? Because most of the fraud here is caught exactly by rules, and a model trained on these files would score near 100% by learning single columns that give the answer away. The split makes the easy part exact and the hard part explainable.

How do you know the agents are not making things up? The reviewer re-queries every cited record and marks each fact. It also checks that every cited ID exists. 71 of 72 facts verified, none contradicted.

What stops label leakage? Rules, features and agent tools all exclude `fraud_flag`, `fraud_type`, `claim_status` and `approved_amount`. Automated tests fail if one reappears. `claim_status` is the clearest trap: it is 100% fraud when "Rejected", because it records the outcome of an investigation that has already happened.

Why Gemini and not another model? It runs on Vertex AI in the same Google Cloud project, so the agents use the project's credentials and credits, with no separate key management. ADK's tools are plain Python functions, which kept the tool layer small.

What does it cost to run? BigQuery stays inside the free tier at this size. Cloud Run scales to zero when idle. The model calls are the only real cost, and only alerts reach them.

Is it secure? The console is private: anonymous requests are refused. The service account can read cleaned data and write only cases and decisions. Agent tools are read-only and use parameterised queries, so nothing an agent produces can modify the data.